Fundamentals of Data Structures & Algorithms

Authors: Dr. Robert Sedgewick & Dr. Alan Turing

Subject: Computer Science

Chapter 1: Asymptotic Analysis and Big-O Notation

Asymptotic analysis is a mathematical framework used to evaluate the runtime and resource consumption of algorithms as the input size n grows indefinitely toward infinity.

1.1 Big-O Notation (O)
Big-O notation establishes a formal asymptotic upper bound on the growth rate of an algorithm. Formally, a function f(n) is in O(g(n)) if there exist positive constants c and n0 such that 0 <= f(n) <= c * g(n) for all n >= n0.

For example, a linear search scanning an unsorted array of size n has a worst-case time complexity of O(n). In contrast, binary search operates on a sorted array by dividing the search space in half at each step, yielding logarithmic complexity O(log n).

1.2 Essential Complexity Classes:
- Constant Time O(1): Accessing an element in an array by index.
- Logarithmic Time O(log n): Binary search in a sorted array.
- Linear Time O(n): Traversing a singly linked list.
- Linearithmic Time O(n log n): Divide-and-conquer sorting like Merge Sort and Heap Sort.
- Quadratic Time O(n^2): Nested comparison loops like Bubble Sort and Insertion Sort.
- Exponential Time O(2^n): Recursive solutions to the traveling salesperson problem.

Chapter 2: Trees and Balanced Search Trees

A tree is a non-linear hierarchical data structure consisting of nodes connected by edges.

2.1 Binary Search Tree (BST) Invariant:
For every node X with key k in a binary search tree:
- Every node in the left subtree of X has a key strictly less than k.
- Every node in the right subtree of X has a key strictly greater than or equal to k.

When a BST remains balanced, basic operations (search, insert, delete) run in O(log n) time.
However, in the degenerate worst case (such as inserting keys in sorted order), the tree degenerates into a linear linked list with O(n) performance.

2.2 AVL Trees and Self-Balancing Invariants:
An AVL tree (named after Adelson-Velsky and Landis) enforces logarithmic balance through rotations.
The balance factor of any node X is defined as: BalanceFactor(X) = height(left) - height(right).
In an AVL tree, the balance factor of every node must be in {-1, 0, 1}.
Whenever an insertion or deletion violates this property, tree rotations restore balance:
- Single Left Rotation (LL): applied when insertion is in the right child of right child.
- Single Right Rotation (RR): applied when insertion is in the left child of left child.
- Double Left-Right Rotation (LR) and Right-Left Rotation (RL): handle zigzag imbalances.

Chapter 3: Graph Traversal and Shortest Paths

Graphs G = (V, E) model networks of relationships between vertices V and connecting edges E.

3.1 Breadth-First Search (BFS) vs Depth-First Search (DFS):
Breadth-First Search traverses vertices in order of increasing distance from the source.
BFS employs a Queue (FIFO) and computes shortest paths in unweighted graphs in O(V + E) time.
Depth-First Search explores paths to maximum depth before backtracking, implemented via recursion or an explicit Stack (LIFO). DFS is widely used in topological sorting and cycle detection.

3.2 Dijkstra's Single-Source Shortest Path Algorithm:
Dijkstra's algorithm finds minimum cost paths from a source vertex to all other vertices in a graph with non-negative edge weights.
With a binary min-priority heap, the time complexity is O((V + E) log V).
Algorithm Execution:
1. Set dist[source] = 0 and dist[v] = infinity for all v != source.
2. Insert (dist[source], source) into the priority queue.
3. While the priority queue is not empty, pop vertex u with minimum distance.
4. For each neighbor v with edge weight w(u, v): if dist[u] + w(u, v) < dist[v], update dist[v] and push the relaxed distance into the priority queue.